

Rust für C/C++ Entwickler

Sicher. Schnell. Ohne Garbage Collector.

| „Rust ist C++ – aber der Compiler übernimmt das, woran du schon einmal gescheitert bist.“

Die Geschichte von Rust

2006 – Der Aufzug: Graydon Hoare kommt nach Hause. Der Aufzug ist wegen eines Software-Absturzes ausgefallen. Er wohnt im **21. Stock**. Auf dem Treppensteigen denkt er: „*Wir können keinen Aufzug bauen, der nicht abstürzt?*“ – Er öffnet seinen Laptop und beginnt Rust zu schreiben.

Jahr	Meilenstein
2009	Mozilla sponsert Rust offiziell
2013	Garbage Collector entfernt – Rust läuft ohne GC
2015	Rust 1.0 – erste stabile Version
2016–2022	7× in Folge beliebteste Sprache (Stack Overflow Survey)
2021	Rust Foundation gegründet (Mozilla, Google, Microsoft, Amazon, ...)
2022	Linux-Kernel nimmt Rust als zweite Systemsprache auf

Warum Rust?

	C/C++	Rust
Performance	✓	✓
Memory Safety	✗ manuell	✓ compile-time
Data Races	✗	✓ unmöglich
Garbage Collector	✗	✗
Zero-Cost Abstraktion	✓	✓
Tooling	😓	✓ cargo

Kein Runtime-Overhead. Kein GC.

Zero-Cost Abstraktion

„What you don't use, you don't pay for. What you do use, you couldn't write better by hand.“ – Bjarne Stroustrup

Hochlesbarer Code – **identisches Assembly** wie die handgeschriebene Schleife:

```
let sum: i32 = vec![1, 2, 3, 4, 5]
    .iter()
    .filter(|&x| x % 2 == 0)
    .map(|&x| x * x)
    .sum();
```

```
// Äquivalentes C – der Compiler erzeugt denselben Maschinencode
int sum = 0;
for (int i = 0; i < 5; i++)
    if (arr[i] % 2 == 0)
        sum += arr[i] * arr[i];
```

Syntax – Basics

C++

```
// Variablen
int x = 5;
const int y = 10;

// Funktion
int add(int a, int b) {
    return a + b;
}

// Struct
struct Point {
    float x;
    float y;
};
```

Rust

```
// Variablen
let x = 5;
let y: i32 = 10; // unveränderlich by default

// Funktion
fn add(a: i32, b: i32) -> i32 {
    a + b // kein return nötig
}

// Struct
struct Point {
    x: f32,
    y: f32,
}
```

Cargo & Tooling

C/C++: `cmake` , `make` , `vcpkg` , `conan` , `gdb` , `valgrind` ...

Rust: Einfach `cargo` .

```
cargo new my_project    # Projekt erstellen
cargo build              # Kompilieren
cargo run                # Bauen + Ausführen
cargo test               # Tests ausführen ← built-in, kein Framework!
cargo add serde          # Dependency hinzufügen
cargo clippy             # Linter
cargo fmt               # Auto-Formatter
cargo doc --open         # Dokumentation generieren
```

`Cargo.toml` = `CMakeLists.txt` + Paketmanager + Test-Runner in einem

Cargo.toml

```
[package]
name     = "my_project"
version  = "0.1.0"
edition  = "2021"

[dependencies]
serde     = { version = "1", features = ["derive"] }
serde_json = "1"
tokio     = { version = "1", features = ["full"] }

[dev-dependencies]
pretty_assertions = "1"

[profile.release]
opt-level = 3
lto       = true
```

Dependencies, Build-Profile und Metadaten – alles in einer Datei.

Ownership – Das Kernkonzept

 examples/01_ownership/main.rs

C++ – du bist verantwortlich:

```
int* ptr = new int(42);
use(ptr);
delete ptr;    // vergisst man → Leak
use(ptr);      // Use-after-free → ✖

// Doppelt freigeben → ✖
delete ptr;
```

Rust – der Compiler überwacht:

```
let s = String::from("hello");
use_string(s);    // Ownership geht über
println!("{}", s); // ✖ COMPILE ERROR

// Scope endet → automatisch gedroppt
// kein delete, kein free, kein GC
{
    let x = String::from("hi");
} // ← x wird hier freigegeben
```

Die drei Ownership-Regeln:

1. Jeder Wert hat genau **einen** Owner
2. Es kann nur **einen** Owner gleichzeitig geben
3. Endet der Scope des Owners → Wert wird **gedroppt**

Ownership – Warum ist das wichtig?

Die häufigsten C/C++ Bugs – in Rust zur Compile-Zeit unmöglich:

Bug	C/C++	Rust
Memory Leak	<code>delete</code> vergessen	✗ unmöglich – automatisch gedroppt
Use-after-free	Pointer nach <code>delete</code> nutzen	✗ Compiler Error
Double-free	<code>delete</code> zweimal aufrufen	✗ Compiler Error
Dangling Pointer	Pointer auf ungültigen Speicher	✗ Compiler Error
Buffer Overflow	Array-Grenzen überschreiten	✗ Panic zur Laufzeit

⚠ Buffer Overflow ist eine Ausnahme – Arraygrenzen sind oft erst zur Laufzeit bekannt. Rust löst das mit einer **Panic** statt stilles UB wie in C++.

Microsoft: **70% aller CVEs** in Windows sind Memory-Safety-Bugs. In Rust wären sie zur Compile-Zeit abgefangen worden.

Borrowing – Referenzen ohne Gefahr

 examples/02_borrowing/main.rs

Ownership übergeben ist oft zu viel – stattdessen **ausleihen**:

```
fn laenge(s: &String) -> usize { // s wird geborgt, nicht übernommen
    s.len()
}

fn main() {
    let s = String::from("hello");
    let n = laenge(&s); // & = borrow
    println!("{}", s); // ✓ s gehört noch main
}
```

Owner bleibt immer Eigentümer – Borrows sind nur **temporäre Leihe**.

Borrowing – Die Regeln

```
let mut s = String::from("hello");

// Mehrere Leser gleichzeitig ✓ – Owner bleibt Eigentümer
let r1 = &s;
let r2 = &s;
let r3 = &s;
println!("{}", r1, r2, r3); // ✓ alle lesen gleichzeitig

// Aber: sobald jemand schreibt → exklusiv
let w = &mut s; // ✓ genau 1 mutable borrow
w.push_str(", world");
// let r4 = &s; // ✗ COMPILE ERROR: nicht gleichzeitig lesen & schreiben
```

Mehrere Leser sind erlaubt, aber nie gleichzeitig mit einem Schreiber → **keine Data Races**. (Compile-Zeit geprüft – kein Runtime-Overhead)

Borrowing – vs C++ Referenzen

C++ – Referenzen, keine Garantien:

```
std::string& get_name() {
    std::string name = "Alice";
    return name; // ✨ Dangling Reference!
}               // name wird hier zerstört

int* dangle() {
    int x = 5;
    return &x; // ✨ Pointer auf Stack-Variable
}
```

Rust – Compiler verhindert Dangling:

```
fn get_name() -> &String {
    let name = String::from("Alice");
    &name // ✖ COMPILER ERROR:
}        // `name` lebt nicht lang genug

// Lösung: Ownership zurückgeben
fn get_name() -> String {
    String::from("Alice") // ✔
}
```

Rust garantiert: Referenzen zeigen immer auf gültigen Speicher.

Lifetimes – Was ist das?

 examples/03_lifetimes/main.rs

Jede Referenz in Rust hat eine **Lifetime** – wie lange sie gültig ist. Meistens inferiert der Compiler sie automatisch:

```
fn main() {
    let s1 = String::from("hello"); // s1 lebt ab hier
    let r;
    {
        let s2 = String::from("world"); // s2 lebt nur im Block
        r = &s2;
    } // s2 wird hier gedroppt
    println!("{}", r); // ❌ COMPILE ERROR: r zeigt auf ungültigen Speicher
}
```

Der Compiler erkennt: `r` würde auf einen bereits zerstörten Wert zeigen – **Dangling Reference zur Compile-Zeit verhindert**, nicht erst beim Absturz.

Lifetimes – Explizite Annotationen

Nötig wenn der Compiler nicht selbst herleiten kann, welche Referenz zurückkommt:

```
// Ohne 'a: Compiler weiß nicht, wie lange der Rückgabewert gültig ist
fn longer<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
// 'a bedeutet: Rückgabewert lebt so lange wie das KÜRZERE der beiden
}
```

```
fn main() {
    let s1 = String::from("long string");
    {
        let s2 = String::from("xyz");
        let result = longer(s1.as_str(), s2.as_str());
        println!("{}", result); // ✓
    }
    // result hier nutzen → ✗ COMPILE ERROR: s2 lebt nicht mehr
}
```

'a ist kein Runtime-Konzept – verschwindet nach der Kompilierung vollständig. Kein Overhead.

Error Handling – Result<T, E>

 examples/04_error_handling/main.rs

In C++ gibt es kein eingebautes `Result` – Fehler werden per Exceptions, Rückgabecodes oder `errno` behandelt, alle ignorierbar. Rust macht Fehler **Teil des Typsystems** – unbehandelte Fehler sind ein Compile Error.

```
use std::fs::File;

fn main() {
    // Result<T,E> zwingt zur Behandlung – kein silent ignore
    match File::open("data.txt") {
        Ok(file) => println!("Geöffnet: {:?}", file),
        Err(e)   => println!("Fehler: {}", e),
    }
}
```

```
// Der ? Operator: Fehler propagieren ohne Boilerplate
fn read_username() -> Result<String, std::io::Error> {
    let s = std::fs::read_to_string("user.txt")?;
    //                                     ^ Err → early return
    Ok(s.trim().to_string())
}
```

`Option<T>` analog – für nullable Werte statt Fehler (`Some` / `None`).

Traits – Interfaces in Rust

 examples/05_traits/main.rs

Traits sind Rusts Antwort auf C++ virtuelle Klassen – aber ohne Vererbung:

C++ – Vererbungshierarchie:

```
class Shape {
public:
    virtual double area() = 0;
};
// Circle IST-EIN Shape
class Circle : public Shape {
    double r;
public:
    Circle(double r) : r(r) {}
    double area() override {
        return M_PI * r * r;
    }
};
```

Rust – Komposition statt Vererbung:

```
trait Area {
    fn area(&self) -> f64;
}
// Circle ist kein "Shape" –
// Circle kann Area berechnen
struct Circle { radius: f64 }

impl Area for Circle {
    fn area(&self) -> f64 {
        std::f64::consts::PI
            * self.radius * self.radius
    }
}
```

In C++ ist `Circle` ein `Shape` – die Hierarchie ist fest verdrahtet. In Rust gibt es keine Basisklasse: `Circle` ist einfach ein Typ, der den `Area`-Trait **implementiert**. Ein Typ kann beliebig viele Traits implementieren, unabhängig voneinander.

Traits können auch **nachträglich** für fremde Typen implementiert werden – z.B. `impl Display for MyStruct`.

Generics – Statischer vs. Dynamischer Dispatch

```
struct Rectangle { width: f64, height: f64 }
impl Area for Rectangle {
    fn area(&self) -> f64 { self.width * self.height }
}

// impl Trait → Compile-Zeit, monomorphisiert (Zero-Cost)
fn print_static(shape: &impl Area) { println!("{}", shape.area()); }

// dyn Trait → Laufzeit, vtable (wie C++ virtual)
fn print_dynamic(shape: &dyn Area) { println!("{}", shape.area()); }

// Heterogene Liste – nur mit dyn möglich
let shapes: Vec<Box<dyn Area>> = vec![
    Box::new(Circle { radius: 3.0 }),
    Box::new(Rectangle { width: 4.0, height: 5.0 }),
];
```

	impl Trait	dyn Trait
Dispatch	statisch (Compile-Zeit)	dynamisch (Laufzeit, vtable)
Performance	Zero-Cost	kleiner Overhead
Heterogene Liste	✗	✓ Vec<Box<dyn Trait>>

Wo wird / kann Rust eingesetzt?



CLI

ripgrep
bat
fd
exa



Web / Cloud

axum
Discord
Cloudflare
npm Registry



WebAssembly

Figma
Google Earth



Embedded

Linux Kernel
Android
Infineon
STM32 / ESP



Desktop

Zed Editor
Gitbutler
Spacedrive

Rust läuft überall – vom Mikrocontroller bis zum Cloud-Datacenter.

DEMO 1 – CLI Tool

```
use std::{env, fs, process};

fn main() {
    let args: Vec<String> = env::args().collect();
    if args.len() < 3 {
        eprintln!("Usage: {} <query> <file>", args[0]);
        process::exit(1);
    }

    let contents = fs::read_to_string(&args[2]).unwrap_or_else(|e| {
        eprintln!("Fehler: {}", e); process::exit(1);
    });

    for (num, line) in search(&args[1], &contents) {
        println!("{:>4}: {}", num, line);
    }
}

// Zero-Copy: Slices zeigen in die Originaldaten – keine Heap-Allokation
fn search<'a>(query: &str, contents: &'a str) -> Vec<(usize, &'a str)> {
    contents.lines().enumerate()
        .filter(|(_, l)| l.contains(query))
        .map(|(i, l)| (i + 1, l))
        .collect()
}
```

Echte Projekte: ripgrep, bat, fd, exa, delta, zoxide, just, mise

DEMO 2 – Web Server (axum)

```
#[derive(Serialize, Deserialize, Clone)]
struct User { id: u32, name: String, age: u32 }

type SharedState = Arc<Mutex<Vec<User>>>;

#[tokio::main]
async fn main() {
    let state: SharedState = Arc::new(Mutex::new(vec![...]));
    let app = Router::new()
        .route("/users", get(get_users).post(create_user))
        .route("/users/:id", get(get_user_by_id))
        .with_state(state);
    axum::serve(TcpListener::bind("0.0.0.0:3000").await.unwrap(), app).await.unwrap();
}

async fn get_user_by_id(State(s): State<SharedState>, Path(id): Path<u32>)
-> Result<Json<User>, StatusCode> {
    s.lock().unwrap().iter().find(|u| u.id == id)
        .cloned().map(Json).ok_or(StatusCode::NOT_FOUND)
}
```

```
curl http://localhost:3000/users
curl -X POST http://localhost:3000/users -d '{"name":"Alice","age":30}'
```

Echte Projekte: **Discord**, Cloudflare Workers, npm Registry

DEMO 3 – Systems / Performance

```
// Zero-Copy Record: Slices zeigen in die Originaldaten
#[derive(Debug)]
struct CsvRecord<'a> { name: &'a str, age: u32, city: &'a str, salary: f64 }

fn parse_record(line: &str) -> Option<CsvRecord<'_>> {
    let mut f = line.splitn(4, ',');
    Some(CsvRecord {
        name: f.next()?.trim(),
        age: f.next()?.trim().parse().ok()?,
        city: f.next()?.trim(),
        salary: f.next()?.trim().parse().ok()?,
    })
}

let start = Instant::now();
let records: Vec<CsvRecord> = csv.lines().skip(1).filter_map(parse_record).collect();
println!("{}", Records in {:.2?}", records.len(), start.elapsed());

// FFI: C-Stdlib direkt aufrufen – zero overhead
extern "C" { fn abs(x: i32) -> i32; }
let result = unsafe { abs(-42) };
```

Echte Projekte: **ripgrep**, rust-analyzer, SWC, Turbopack, Deno

DEMO 4 – WebAssembly

```
rustup target add wasm32-unknown-unknown
cargo build --target wasm32-unknown-unknown --release
wasm-bindgen --out-dir ./out --target web target/.../pkg.wasm
```

```
use wasm_bindgen::prelude::*;

#[wasm_bindgen]
pub fn fibonacci(n: u32) -> u64 { /* ... */ }

// Rust-Struct wird zur JavaScript-Klasse
#[wasm_bindgen]
pub struct Counter { value: i32, step: i32 }

#[wasm_bindgen]
impl Counter {
    #[wasm_bindgen(constructor)]
    pub fn new(step: i32) -> Counter { Counter { value: 0, step } }
    pub fn increment(&mut self) { self.value += self.step; }
    pub fn get(&self) -> i32 { self.value }
}
```

```
// JavaScript – native Typen, keine Wrapper
const c = new Counter(5);
c.increment();
console.log(c.get(), fibonacci(10)); // → 5, 55
```

Echte Projekte: **Figma** (Rendering Engine), Google Earth, Cloudflare Workers

DEMO 5 – Embedded / no_std

```
rustup target add thumbv7em-none-eabihf
cargo build --target thumbv7em-none-eabihf
probe-rs run --chip STM32F411RETx
```

```
#![no_std]    // kein std, kein Heap, kein OS
#![no_main]

#[embassy_executor::main]
async fn main(spawner: Spawner) {
    let p = embassy_stm32::init(Default::default());

    // Typsicher: falscher Pin → Compile Error, nicht Runtime Error
    let mut led = Output::new(p.PB7, Level::High, Speed::Low);

    // Zweiter Task – kein FreeRTOS, kein RTOS nötig!
    spawner.spawn(blink_fast(led2)).unwrap();

    loop {
        led.set_high();
        Timer::after(Duration::from_millis(500)).await; // async – kein Busy-Wait!
        led.set_low();
        Timer::after(Duration::from_millis(500)).await;
    }
}
```

Echte Projekte: **Linux-Kernel** (seit 6.1!), Android (Binder), Azure Sphere, Infineon Automotive

DEMO 6 – Web UI (Leptos)

```
use leptos::*;

// Reaktive Komponente – kompiliert zu WASM, kein JS-Framework nötig
#[component]
fn Counter() -> impl IntoView {
    let (count, set_count) = create_signal(0);
    let doubled = move || count() * 2;

    view! {
        <div>
            <p>"Count: " {count} " (doubled: " {doubled} ")"</p>
            <button on:click=move |_| set_count.update(|n| *n -= 1)>"- "</button>
            <button on:click=move |_| set_count.set(0)>"Reset"</button>
            <button on:click=move |_| set_count.update(|n| *n += 1)>"+"</button>
        </div>
    }
}

fn main() { mount_to_body(|| view! { <Counter /> }) }
```

Kein Virtual DOM. Direktes WASM → DOM Update.

Template-Fehler sind **Compile-Fehler**.

Echte Projekte: **Zed** (eigenes GPU-UI-Framework), Leptos, Dioxus

DEMO 7 – Desktop App (Tauri)

```
#[derive(Serialize, Deserialize, Clone)]
struct Note { id: u32, title: String, content: String }

// Rust-Funktionen direkt aus JavaScript aufrufbar – typsicher
#[command]
fn get_notes(state: State<AppState>) -> Vec<Note> {
    state.notes.lock().unwrap().clone()
}

#[command]
fn add_note(title: String, content: String, state: State<AppState>) -> Note { /* ... */ }
```

```
// JavaScript Frontend (React / Svelte / Vanilla):
import { invoke } from '@tauri-apps/api/tauri';
const notes = await invoke('get_notes');
const note = await invoke('add_note', { title: 'Hey', content: '🦀' });
```

	Electron	Tauri
Bundle-Grösse	~150 MB	~8 MB
RAM-Verbrauch	~200 MB	~30 MB

Echte Projekte: **Zed**, Gitbutler, Spacedrive

Zusammenfassung

Konzept	Rust-Lösung
Memory Safety	Ownership + Borrow Checker (compile-time)
Fehlerbehandlung	<code>Result<T,E></code> + <code>?</code> Operator
Abstraktion	Traits – statisch & dynamisch (Zero-Cost)
Tooling	<code>cargo</code> – alles in einem
Einsatzgebiete	CLI • Web • WASM • Embedded • Desktop • UI

Wo anfangen?

```
# Rust installieren
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh

# Interaktiv lernen – kleine Übungsaufgaben im Terminal
cargo install rustlings && rustlings

# VSCode Extension
rust-analyzer
```

Lesen:

-  [The Rust Book](#) – kostenlos online
-  [Programming Rust](#) – O'Reilly, ideal für C++-Hintergrund
-  [Rustonomicon](#) – für unsafe Rust

Fragen?